

Práctica 1: Patrones de Diseño

Bloque I

Modelado y Diseño Avanzado de Software

Grado en Ingeniería Informática

Escuela Politécnica Superior

Universidad de Córdoba

Equipo 5:

Álvaro Jurado Pozo (i32jupoa@uco.es)
Ádam Gálvez Redondo (i32garea@uco.es)
Marcos Hidalgo Moreno (i32himom@uco.es)

Profesora:

Aurora Ramírez Quesada

Curso 2025/2026

Índice

1. Introducción	3
2. Semana 1: Patrón Abstract Factory (Sistema de Pedidos)	4
2.1. Decisiones de Diseño	4
2.2. Diagrama de Clases	4
3. Semana 2: Patrón Adapter (Sistema de Maquetación)	5
3.1. Decisiones de Diseño	5
3.2. Diagrama de Clases	5
4. Semana 3: Patrón Composite (Gasto Energético)	6
4.1. Decisiones de Diseño	6
4.2. Diagrama de Clases	6
5. Semana 4: Patrón Bridge (Catálogo de Proveedores)	7
5.1. Decisiones de Diseño	7
5.2. Diagrama de Clases	7
6. Semana 5: Patrón Facade (Buscador de Viajes)	8
6.1. Decisiones de Diseño	8
6.2. Diagrama de Clases	8
7. Conclusión	9

Índice de figuras

1.	GitHub de la práctica.	3
2.	Diagrama de Clases de la solución Abstract Factory.	4
3.	Diagrama de Clases de la solución Adapter.	5
4.	Diagrama de Clases de la solución Composite.	6
5.	Diagrama de Clases de la solución Bridge.	7
6.	Diagrama de Clases de la solución Facade.	8

1. Introducción

En este informe se detalla la aplicación de cinco patrones de diseño fundamentales (Abstract Factory, Adapter, Composite, Bridge y Façade) para resolver diversos problemas arquitectónicos planteados en los enunciados.

Enlace al repositorio GitHub con el código fuente:

https://github.com/i32jupoa/Practica1_equipo5.git

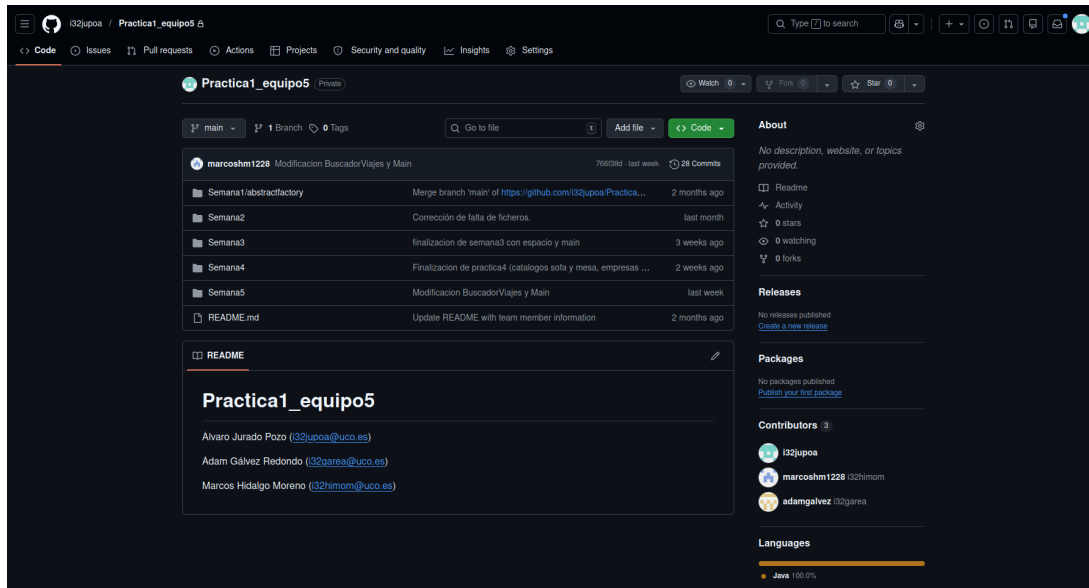


Figura 1: GitHub de la práctica.

2. Semana 1: Patrón Abstract Factory (Sistema de Pedidos)

2.1. Decisiones de Diseño

Para resolver el problema del sistema de pedidos del restaurante, se ha aplicado el patrón creacional Abstract Factory. El problema principal requería crear familias de productos (Menú Semanal y Menú de Temporada) que presentaban variaciones significativas dependiendo de si el consumo se realizaba en el local o era un pedido para llevar.

Las decisiones adoptadas en el diseño son:

- **Factoría Abstracta:** Se define la interfaz `FactoriaAbstracta` que declara los métodos de creación para los productos abstractos (`crearMenuSemanal()` y `crearMenuTemporada()`).
- **Factorías Concretas:** Se implementan `FactoriaRestaurante` y `FactoriaParaLlevar`. La lógica de negocio específica se ha encapsulado aquí. Por ejemplo, `FactoriaParaLlevar` implementa internamente el incremento del 2% en los precios y omite la instanciación del postre en la composición del menú semanal.
- **Productos:** Se ha diseñado una clase base `Menu` de la que heredan los menús concretos, los cuales están compuestos por una lista de objetos de la clase `Plato`. Esto otorga flexibilidad para poder iterar sobre los platos y calcular dinámicamente el coste total del menú en base a la familia seleccionada.

2.2. Diagrama de Clases

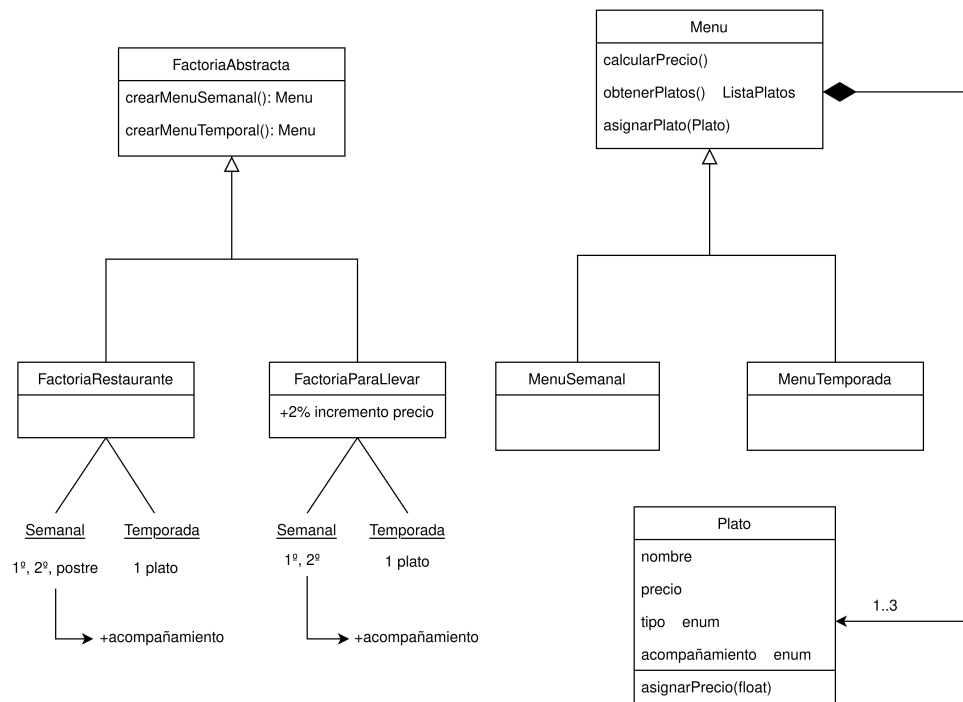


Figura 2: Diagrama de Clases de la solución Abstract Factory.

3. Semana 2: Patrón Adapter (Sistema de Maquetación)

3.1. Decisiones de Diseño

Ante la necesidad de dotar de funcionalidades avanzadas a un sistema de maquetación basándose en un código legado con operaciones primitivas, la solución óptima implementada es el patrón estructural Adapter.

Las decisiones clave adoptadas son:

- **Target e Interfaz Requerida:** Se ha creado la interfaz `MaquetadorAvanzado` que define exactamente los métodos que el cliente espera utilizar (unir, combinar y separar ficheros).
- **El Adaptador (`AdaptadorMaquetador`):** Actúa como puente. Implementa la interfaz `MaquetadorAvanzado` pero utiliza de forma interna la composición para mantener una instancia del `MaquetadorBasico` (el Adapter).
- **Traducción de lógica compleja:** El adaptador descompone las operaciones complejas. Por ejemplo, para combinar ficheros intercalando párrafos, el adaptador itera sobre los índices proporcionados, invoca repetidamente a la función `extraerParrafo` del maquetador básico, y utiliza `anadirTextoAlFinal` para ir construyendo el archivo destino. Esto cumple el requisito de no modificar el código existente de `MaquetadorBasico`.

3.2. Diagrama de Clases

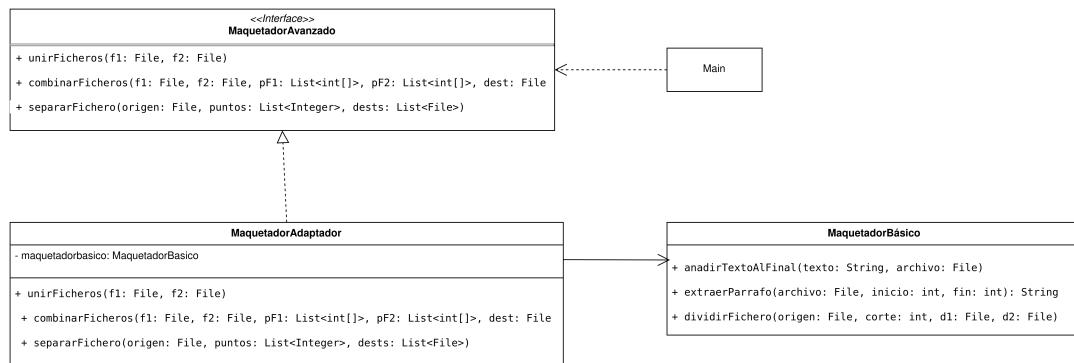


Figura 3: Diagrama de Clases de la solución Adapter.

4. Semana 3: Patrón Composite (Gasto Energético)

4.1. Decisiones de Diseño

El cálculo del gasto energético en una estructura jerárquica (Campus → Edificios → Salas → Aparatos) es el escenario de uso clásico para el patrón estructural Composite.

La justificación de las decisiones es la siguiente:

- **Componente Base (ComponenteEnergetico):** Se estableció una interfaz común con los métodos `getConsumoEstimado()` y `getCosteEstimado()`. Esto permite al cliente tratar de forma completamente idéntica a un simple proyector que a un edificio entero.
- **Nodos Hoja (AparatoElectrico):** Almacenan los datos primitivos (consumo por hora y horas de uso). No pueden contener a otros componentes y realizan el cálculo matemático base.
- **Nodos Compuestos (Espacio):** Mantienen una lista de hijos de tipo `ComponenteEnergetico`. Su implementación del cálculo de coste es una operación recursiva que delega la suma a sus hijos.
- **Flexibilidad Topológica:** Dado que un objeto de tipo `Espacio` admite cualquier `ComponenteEnergetico`, se ha podido satisfacer fácilmente el requisito de incluir aparatos directamente en los pasillos de un edificio (sin estar dentro de una sala), demostrando la robustez de este patrón.

4.2. Diagrama de Clases

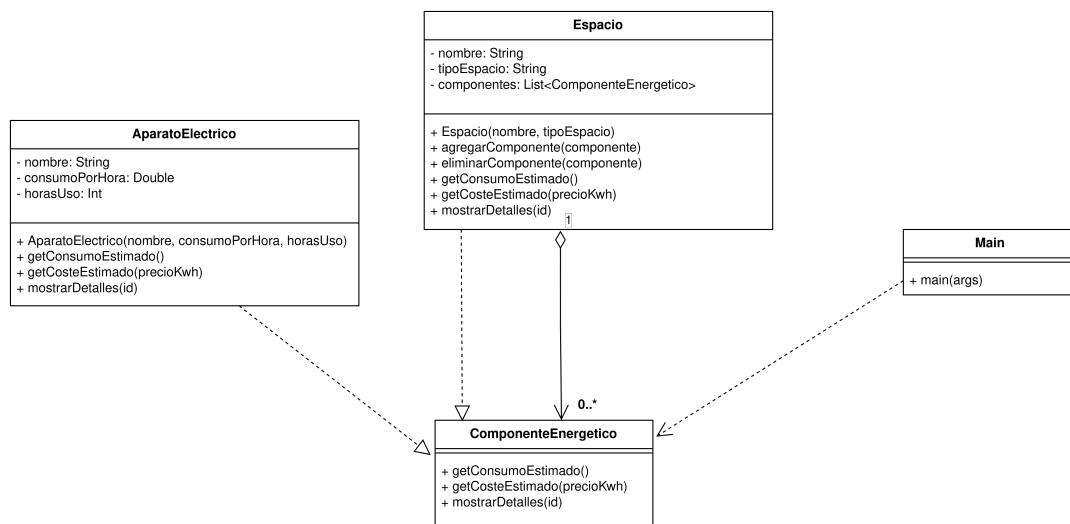


Figura 4: Diagrama de Clases de la solución Composite.

5. Semana 4: Patrón Bridge (Catálogo de Proveedores)

5.1. Decisiones de Diseño

El problema planteaba dos ejes de cambio: por un lado, los distintos tipos de catálogos y filtros que necesita la empresa (Abstracción) y, por otro, las diferentes empresas proveedoras (Implementación). Para evitar una explosión combinatoria de subclases, se ha utilizado el patrón Bridge.

Decisiones arquitectónicas:

- **Desacoplamiento de Jerarquías:** Se ha creado la clase abstracta `Catalogo` (que se especializa en `CatalogoMesas` y `CatalogoSofas`) y la interfaz `Proveedor` (implementada por `EmpresaA`, `EmpresaB` y `EmpresaC`).
- **El Puente (Composición):** La jerarquía de abstracción mantiene una lista de referencias a la interfaz `Proveedor`. Esto permite configurar el catálogo en tiempo de ejecución. Por ejemplo, al buscar mesas, el sistema inyecta las empresas B y C en el `CatalogoMesas`.
- **Agregación de Stock de forma segura:** Para cumplir el requisito de fusionar inventarios de un mismo producto en distintas empresas sin alterar los datos originales de cada proveedor, se ha incorporado el patrón `Prototype` (método `clonar()`) en la clase `Producto`, permitiendo sumar las unidades temporalmente en las búsquedas avanzadas del catálogo.

5.2. Diagrama de Clases

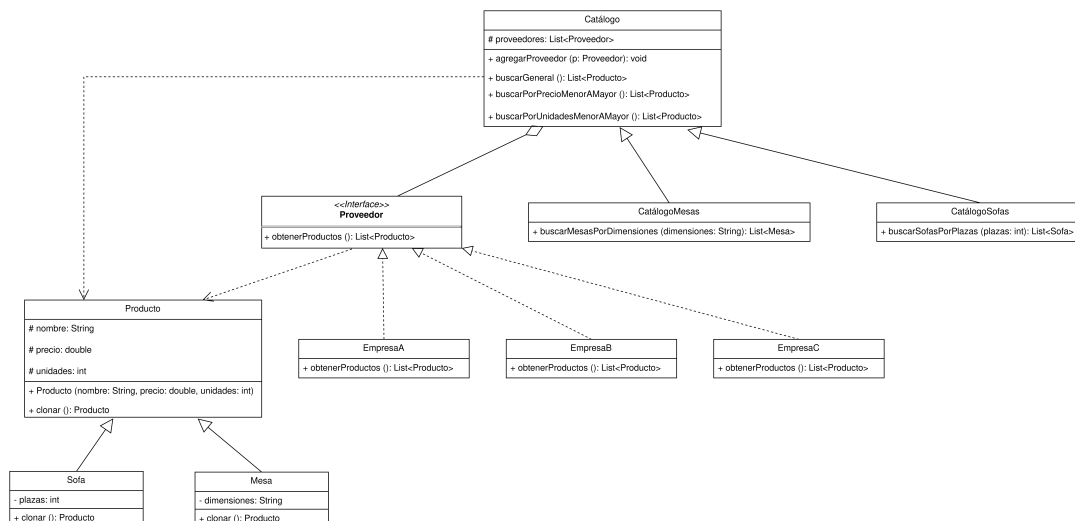


Figura 5: Diagrama de Clases de la solución Bridge.

6. Semana 5: Patrón Facade (Buscador de Viajes)

6.1. Decisiones de Diseño

El sistema de la agencia de viajes presentaba una gran complejidad para el código cliente, el cual debía orquestar llamadas a sistemas dispares (alojamiento y transporte) y lidiar con las interfaces incompatibles de múltiples oficinas de turismo. Se ha solucionado mediante el patrón Facade.

Decisiones fundamentales:

- **Punto de Entrada Único:** Se ha diseñado la clase `BuscadorViajesFacade` que expone un único método simplificado (`buscarViajeCompleto`) que recibe los datos esenciales (fechas, origen y destino).
- **Orquestación y Ocultación:** El cliente (clase `Main`) desconoce por completo la existencia de los subsistemas. La fachada instancia `ServicioAlojamiento` y `ServicioTransporte`, y coordina sus respuestas.
- **Adaptación de Interfaces Heterogéneas:** El patrón Facade absorbe la lógica condicional compleja referida a las oficinas de turismo. Si el destino es Madrid, la fachada extrae solo la fecha requerida para la API de `OficinaTurismoMadrid`; si es Córdoba, proporciona el tipo de evento y el rango de fechas que exige `OficinaTurismoCordoba`.

6.2. Diagrama de Clases

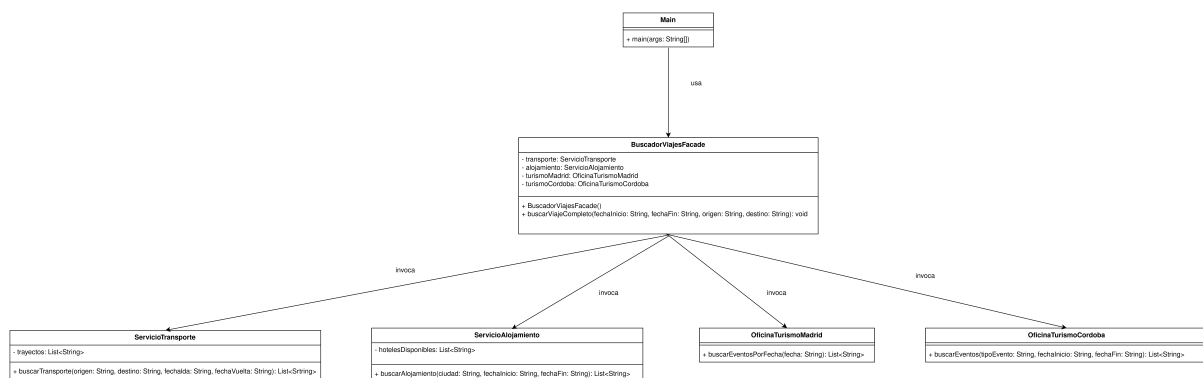


Figura 6: Diagrama de Clases de la solución Facade.

7. Conclusión

A lo largo de este primer bloque de prácticas se ha comprobado cómo la correcta aplicación de los patrones de diseño (creacionales y estructurales) mejora drásticamente la calidad, mantenibilidad y escalabilidad del código. El desarrollo del proyecto en Java ha permitido observar cómo principios clave del diseño de software (como favorecer la composición sobre la herencia, programar hacia interfaces y el principio de responsabilidad única) se materializan a través de estos patrones estandarizados, solucionando problemas recurrentes de forma elegante y eficiente.

Referencias

[1] <https://gemini.google.com/app>